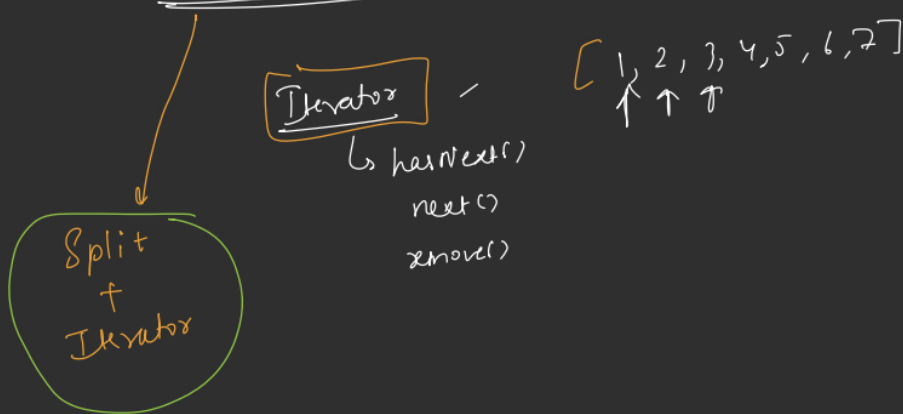


Parallel Stream

- Split data into chunks
- Assign chunk to Thread
- Process independently.
- Combine Result.



Split Iterator



Split Iterator

- ① Traverse the elements
- ② Decompose the source into parts
- ③ Describe the source

`[1, 2, 3, 4, 5]`

Split Iterators

tryAdvance() // traversal
trySplit()



1, 2, 3, 4, 5, 6, 7, 8, 9, 10

stackless

1 2 3 4 5

6 7 8 9 10

1 2 3

4 5

6 7 8

9 10

sorted

(1, 2, 3) (4, 5) (6, 7, 8) (9, 10)

distinct

list.parallelStream()

✓ filter(x → x > 10)

✓ map(x → x * 2)

• sorted()

• distinct()

• forEach(System.out::println);

T1 T2

T3 T4

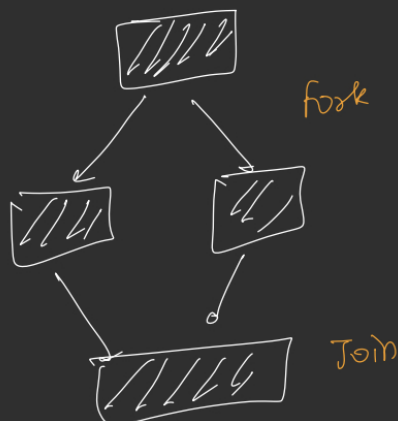
T5

Java

↳ Size of data

↳ Complexity of problem

↳ Threads availability



When parallel streams can help?

LinkedList

- ① Dataset is Huge (Millions)
- ② CPU intensive work
- ③ Stateless operations
- ④ Optimised Data structure for parallel streams

Arrays

ArrayList

When NOT to use

↳ Dataset small

Overhead > Performance

Stateful

Shared Mutable resource

List<Integer> list; = (1, 2, 3, 4, 5)

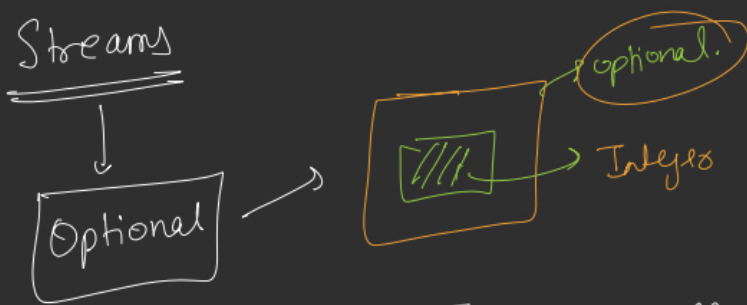
List<Integer> result;

Race Condition

list.parallelStream()

forEach(x → result.add(x));

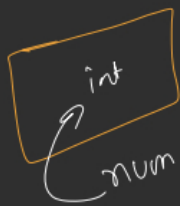
Streams



Optional <String> name = getName();

name.get()
- orElse()

Primitive Optional classes



- ↳ Optional Int
- ↳ Optional Double.
- ↳ Optional Long

Optional Int num = getIntStream() = filter()
- map()
- max() / arg() OptionalDouble
num.getAsInt() OptionalInt